



DALHOUSIE UNIVERSITY

CSCI 5408: Data Management, Warehousing and Analytics

Lab - 05

Banner ID: B01026328

Name: Parva Mineshbhai Patel

Date: 2025 – 03 – 09

GitLab Assignment Link: <https://git.cs.dal.ca/parva/>

CSCI5408_W25_B01026328_ParvaMineshbhai_Patel

Contents

1) Set up an Apache Spark Cluster on your GCP instance(Using Dataproc).....	1
2) Java Program.....	4
3) Execution of JAR file.....	6
4) SSH output.....	7
5) References.....	9

1) Set up an Apache Spark Cluster on your GCP instance(Using Dataproc)

Step 1: After Enabling the API for the Dataproc in the Google Cloud Platform after that using search, I selected the Cloud Dataproc and then click the create cluster.

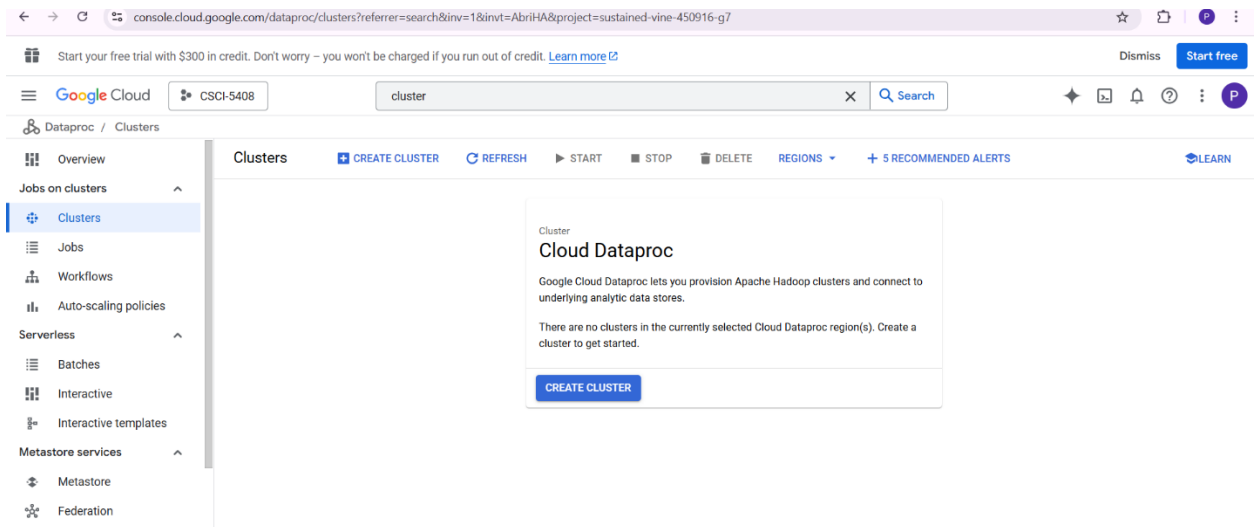


Figure 1: Cloud Dataproc and Create Cluster

Step 2: I have clicked the cluster on Compute Engine.

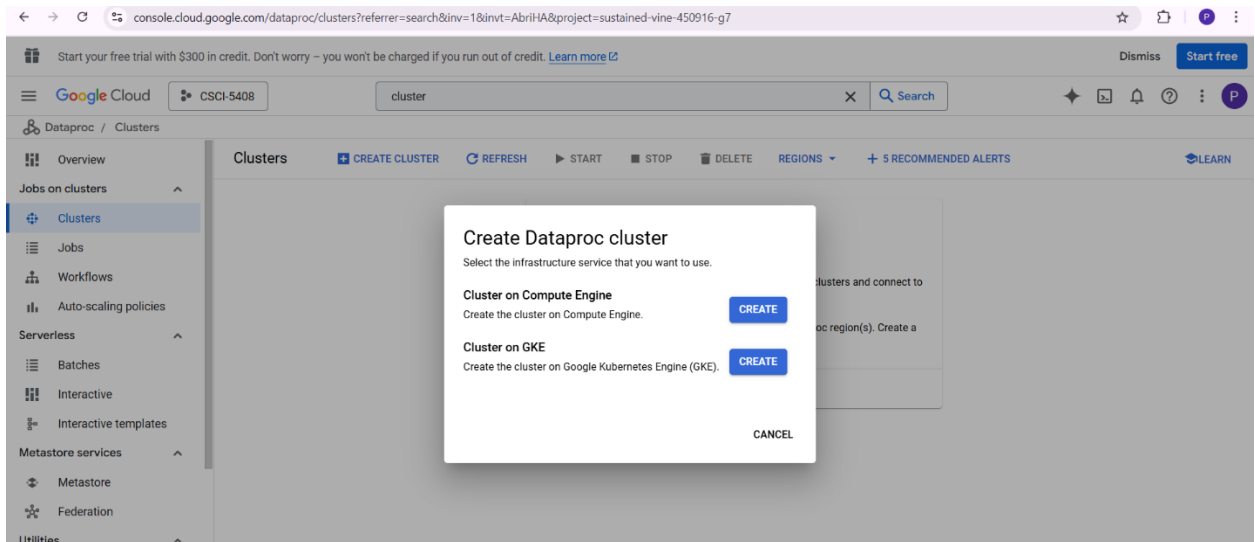


Figure 2: Create Cluster on Compute Engine

Step 3: Set up cluster like name, location and cluster type.

console.cloud.google.com/dataproc/clustersAdd?computeInfra=goe&inv=AbRh_g&project=sustained-vine-450916-g7

Start your free trial with \$300 in credit. Don't worry – you won't be charged if you run out of credit. [Learn more](#)

Google Cloud CSC1-5408 clusters

Dataproc / Clusters / Configure cluster

Overview

Jobs on clusters

Clusters

Jobs

Workflows

Auto-scaling policies

Serverless

Batches

Interactive

Interactive templates

Metastore services

Metastore

Federation

Utilities

Create a Dataproc cluster on Compute Engine

- Set up cluster
Begin by providing basic information.
- Configure nodes (optional)
Change node compute and storage capabilities.
- Customise cluster (optional)
Add cluster properties, features and actions.
- Manage security (optional)
Change access, encryption and security settings.

CREATE CANCEL

EQUIVALENT COMMAND LINE

EQUIVALENT REST

Name

Cluster name *
lab8

Location

Region *
us-central1

Zone *
us-central1-a

Cluster type

☒ Standard (1 master, N workers)

☐ Single Node (1 master, 0 workers)
Provides one node that acts as both master and worker. Good for proof-of-concept or small-scale processing.

☐ High availability (3 masters, N workers)
Hadoop high availability mode provides uninterrupted YARN and HDFS operations despite single-node failures or reboots.

Versioning

Figure 3: Creating Cluster

Step 4: Configuring nodes like reduced the disk size from 200 GB to 50 GB.

Start your free trial with \$300 in credit. Don't worry – you won't be charged if you run out of credit. [Learn more](#)

Google Cloud CSC1-5408 clusters

Dataproc / Clusters / Configure cluster

Overview

Jobs on clusters

Clusters

Jobs

Workflows

Auto-scaling policies

Serverless

Batches

Interactive

Interactive templates

Metastore services

Metastore

Federation

Utilities

Create a Dataproc cluster on Compute Engine

- Set up cluster
Begin by providing basic information.
- Configure nodes (optional)
Change node compute and storage capabilities.
- Customise cluster (optional)
Add cluster properties, features and actions.
- Manage security (optional)
Change access, encryption and security settings.

CREATE CANCEL

EQUIVALENT COMMAND LINE

EQUIVALENT REST

Manager node

Contains the YARN Resource Manager, HDFS NameNode and all job drivers.

☒ General purpose ☐ Compute-optimised ☐ Memory-optimised ☐ GPUs

Machine types for common workloads, optimised for cost and flexibility

Series
E2

CPU platform selection based on availability

Machine type
e2-standard-2 (2 vCPU, 1 core, 8 GB memory)

vCPU
2

Memory
8 GB

CPU PLATFORM AND GPU

Primary disk size *
50 GB

Primary disk type *
Balanced Persistent Disk

Figure 4: Manager node

Step 5: Configuring nodes and set the E2 series and reduced the disk size from 200GB to 50GB.

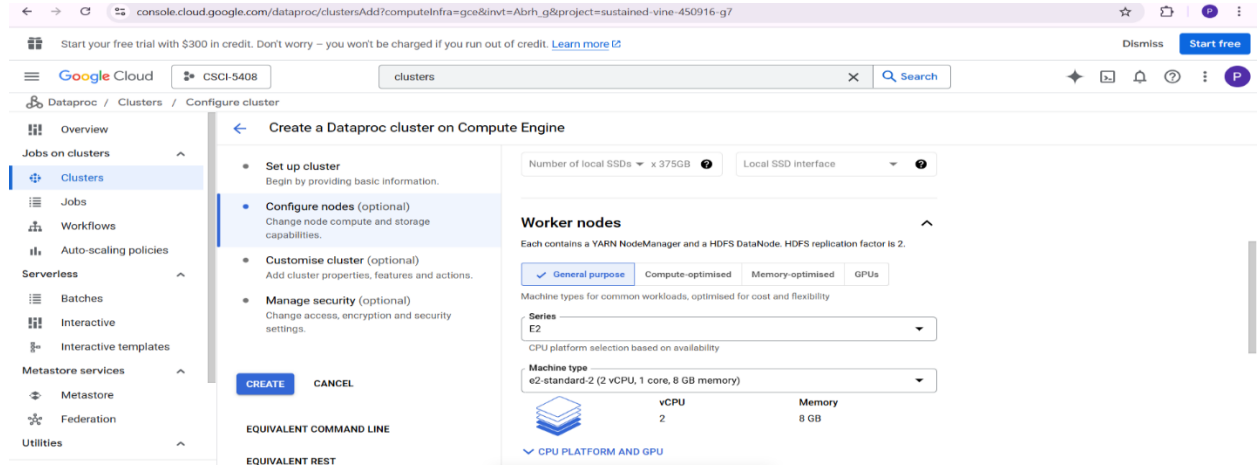


Figure 5: Worker Node

Step 6: Now the cluster is running as well as nodes.

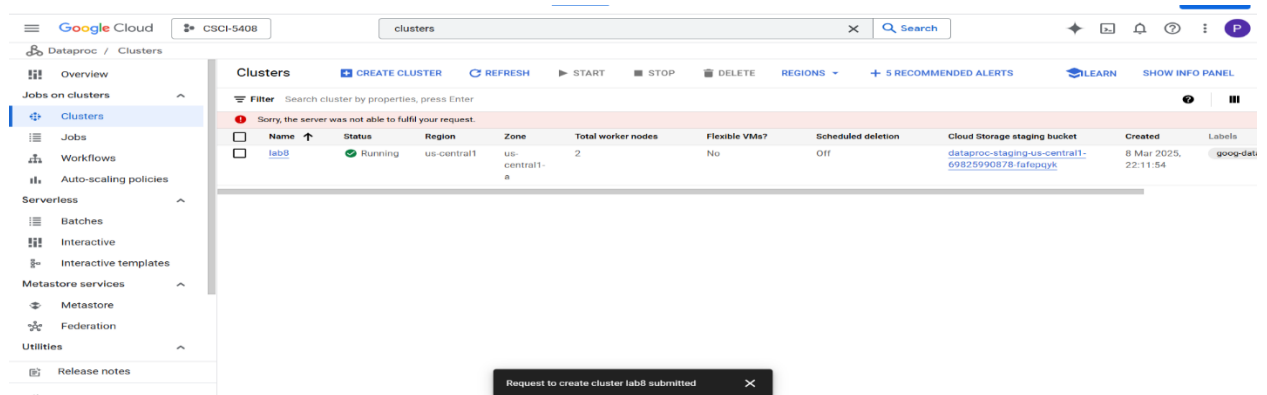


Figure 6: Cluster is Running

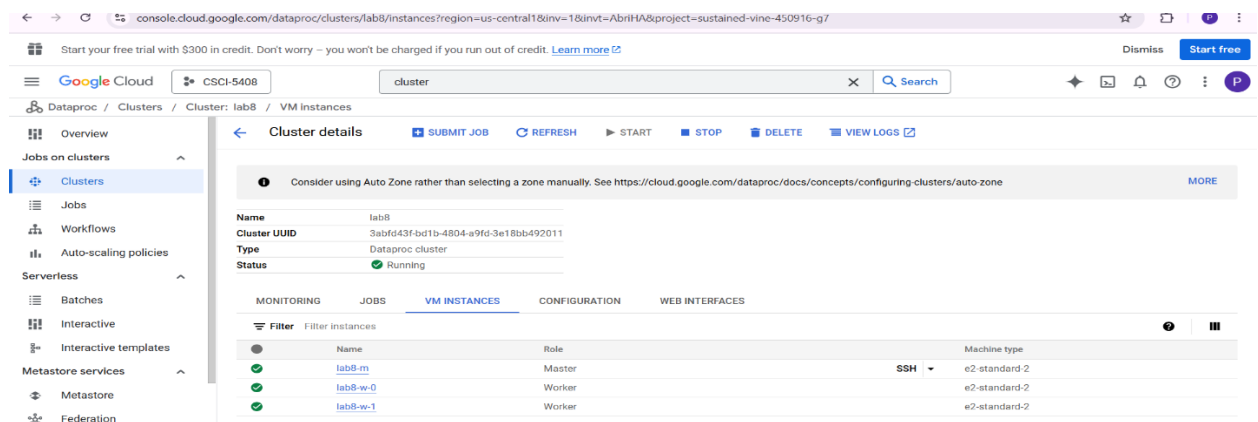
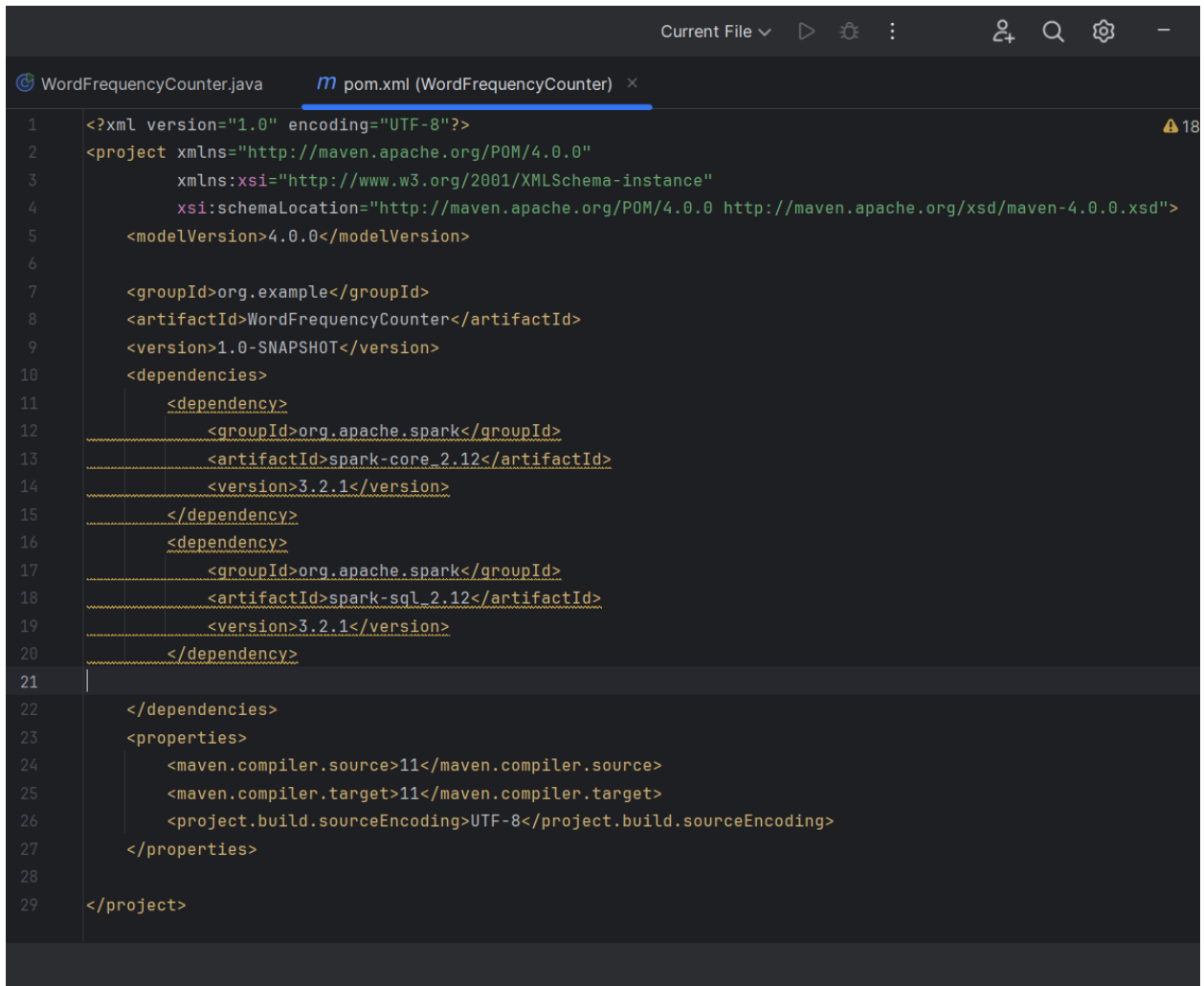


Figure 7: VM Instance

2) Java Program

I added Dependencies for the spark core and spark sql in pom.xml.

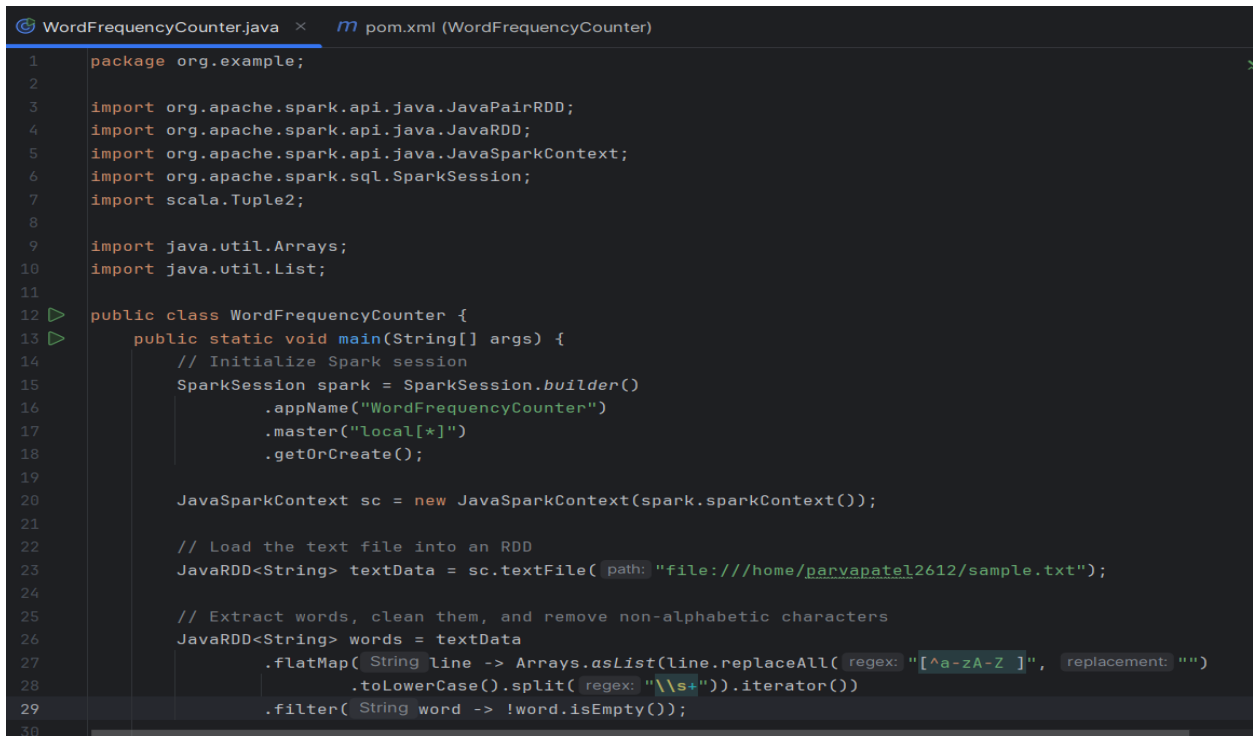


```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>org.example</groupId>
8     <artifactId>WordFrequencyCounter</artifactId>
9     <version>1.0-SNAPSHOT</version>
10    <dependencies>
11        <dependency>
12            <groupId>org.apache.spark</groupId>
13            <artifactId>spark-core_2.12</artifactId>
14            <version>3.2.1</version>
15        </dependency>
16        <dependency>
17            <groupId>org.apache.spark</groupId>
18            <artifactId>spark-sql_2.12</artifactId>
19            <version>3.2.1</version>
20        </dependency>
21    </dependencies>
22    <properties>
23        <maven.compiler.source>11</maven.compiler.source>
24        <maven.compiler.target>11</maven.compiler.target>
25        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
26    </properties>
27
28 </project>
```

Figure 8: Spark Dependencies

Explanation of code

The WordFrequencyCounter Java program uses Apache Spark to count the frequency of each word in a text file. It first initializes a Spark session and loads the input text file into RDD.



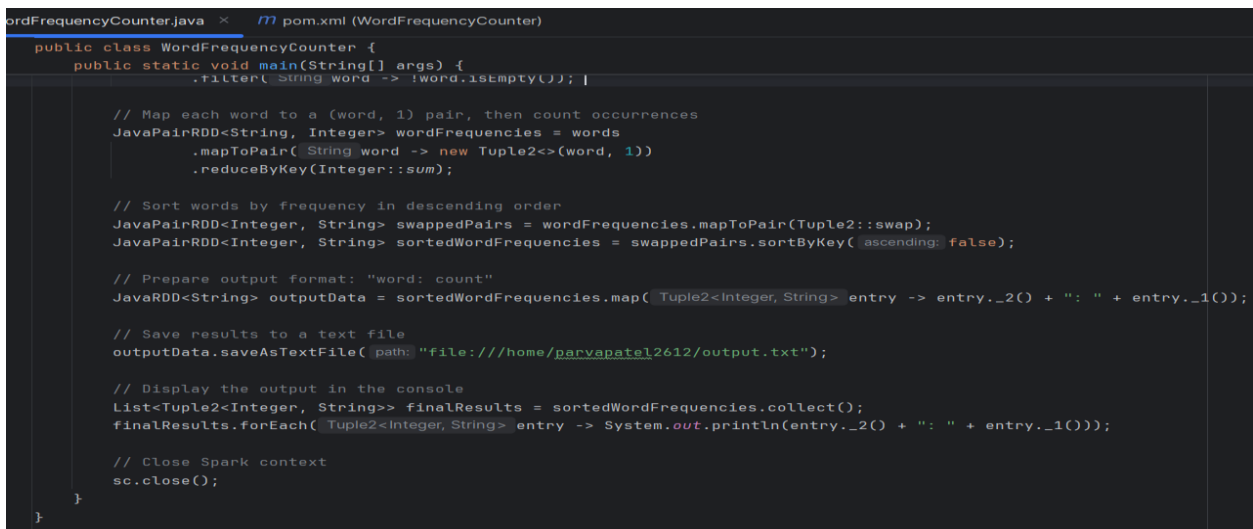
```

1 package org.example;
2
3 import org.apache.spark.api.java.JavaPairRDD;
4 import org.apache.spark.api.java.JavaRDD;
5 import org.apache.spark.api.java.JavaSparkContext;
6 import org.apache.spark.sql.SparkSession;
7 import scala.Tuple2;
8
9 import java.util.Arrays;
10 import java.util.List;
11
12 public class WordFrequencyCounter {
13     public static void main(String[] args) {
14         // Initialize Spark session
15         SparkSession spark = SparkSession.builder()
16             .appName("WordFrequencyCounter")
17             .master("local[*]")
18             .getOrCreate();
19
20         JavaSparkContext sc = new JavaSparkContext(spark.sparkContext());
21
22         // Load the text file into an RDD
23         JavaRDD<String> textData = sc.textFile(path: "file:///home/parvapatel2612/sample.txt");
24
25         // Extract words, clean them, and remove non-alphabetic characters
26         JavaRDD<String> words = textData
27             .flatMap(line -> Arrays.asList(line.replaceAll(regex: "[^a-zA-Z ]", replacement: "")
28                 .toLowerCase().split(regex: "\\s+")))
29             .filter(word -> !word.isEmpty());
30

```

Figure 9: Java program uses the spark session and RDD

Moreover, the program then processes the text by removing non-alphabetic characters, converting all words to lowercase, and splitting the text into individual words.



```

// Map each word to a (word, 1) pair, then count occurrences
JavaPairRDD<String, Integer> wordFrequencies = words
    .mapToPair(word -> new Tuple2<>(word, 1))
    .reduceByKey(Integer::sum);

// Sort words by frequency in descending order
JavaPairRDD<Integer, String> swappedPairs = wordFrequencies.mapToPair(Tuple2::swap);
JavaPairRDD<Integer, String> sortedWordFrequencies = swappedPairs.sortByKey(ascending: false);

// Prepare output format: "word: count"
JavaRDD<String> outputData = sortedWordFrequencies.map(entry -> entry._2() + ": " + entry._1());

// Save results to a text file
outputData.saveAsTextFile(path: "file:///home/parvapatel2612/output.txt");

// Display the output in the console
List<Tuple2<Integer, String>> finalResults = sortedWordFrequencies.collect();
finalResults.forEach(entry -> System.out.println(entry._2() + ": " + entry._1()));

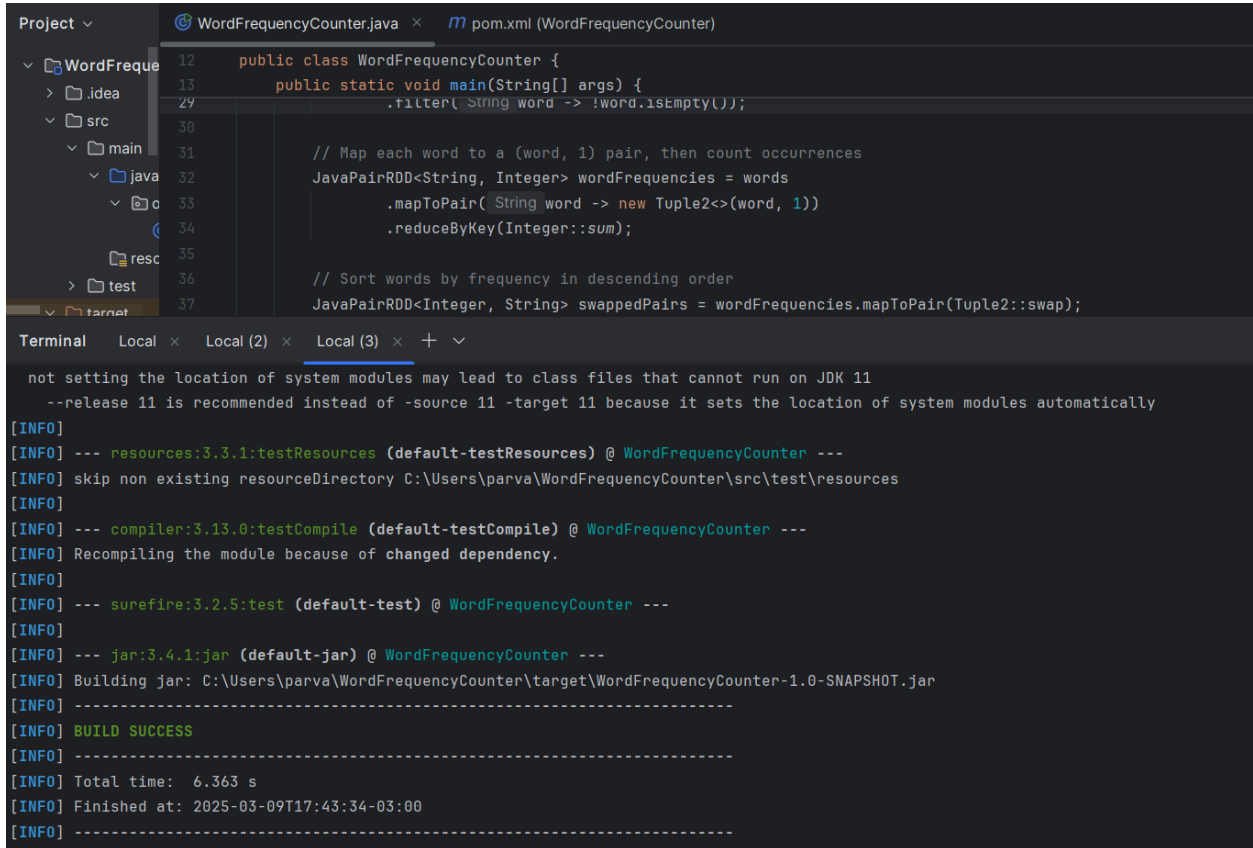
// Close Spark context
sc.close();
}
}

```

Figure 10: Java program for descending order and word count

The word-frequency pairs are then sorted in descending order based on frequency, and the results are saved to an output file. Finally, the program prints the sorted word frequencies.

3) Execution of the JAR file



The screenshot displays an IDE with the `WordFrequencyCounter.java` file open. The code defines a `WordFrequencyCounter` class with a `main` method that processes a list of words using Apache Spark. The `main` method filters out empty strings, maps each word to a `(word, 1)` pair, reduces by key to count occurrences, sorts by frequency in descending order, and finally swaps the keys and values of the resulting `JavaPairRDD`.

```
12 public class WordFrequencyCounter {
13     public static void main(String[] args) {
29         .filter( >String word -> !word.isEmpty());
30
31         // Map each word to a (word, 1) pair, then count occurrences
32         JavaPairRDD<String, Integer> wordFrequencies = words
33             .mapToPair( >String word -> new Tuple2<>(word, 1))
34             .reduceByKey(Integer::sum);
35
36         // Sort words by frequency in descending order
37         JavaPairRDD<Integer, String> swappedPairs = wordFrequencies.mapToPair(Tuple2::swap);
```

The terminal window shows the output of the `mvn clean package` command. It includes warnings about JDK 11 and release 11, followed by several informational messages from Maven and Surefire. The build is successful, and the JAR file `WordFrequencyCounter-1.0-SNAPSHOT.jar` is created in the `target` directory.

```
not setting the location of system modules may lead to class files that cannot run on JDK 11
--release 11 is recommended instead of -source 11 -target 11 because it sets the location of system modules automatically
[INFO] --- resources:3.3.1:testResources (default-testResources) @ WordFrequencyCounter ---
[INFO] skip non existing resourceDirectory C:\Users\parva\WordFrequencyCounter\src\test\resources
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ WordFrequencyCounter ---
[INFO] Recompiling the module because of changed dependency.
[INFO] --- surefire:3.2.5:test (default-test) @ WordFrequencyCounter ---
[INFO] --- jar:3.4.1:jar (default-jar) @ WordFrequencyCounter ---
[INFO] Building jar: C:\Users\parva\WordFrequencyCounter\target\WordFrequencyCounter-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time: 6.363 s
[INFO] Finished at: 2025-03-09T17:43:34-03:00
```

Figure 11: Execution of JAR Successful

I have written the `mvn clean package` command to create the jar file and it is created successfully.

4) SSH Output

Uploaded WordFrequencyCounter jar file using SSH.

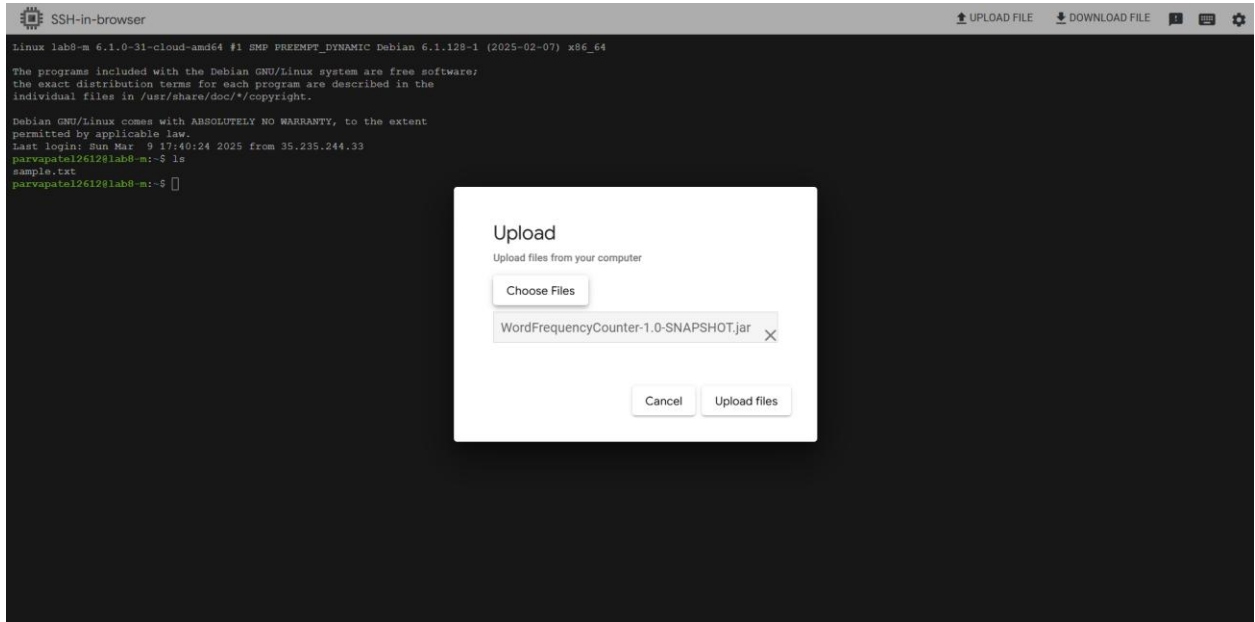


Figure 12: Upload JAR file

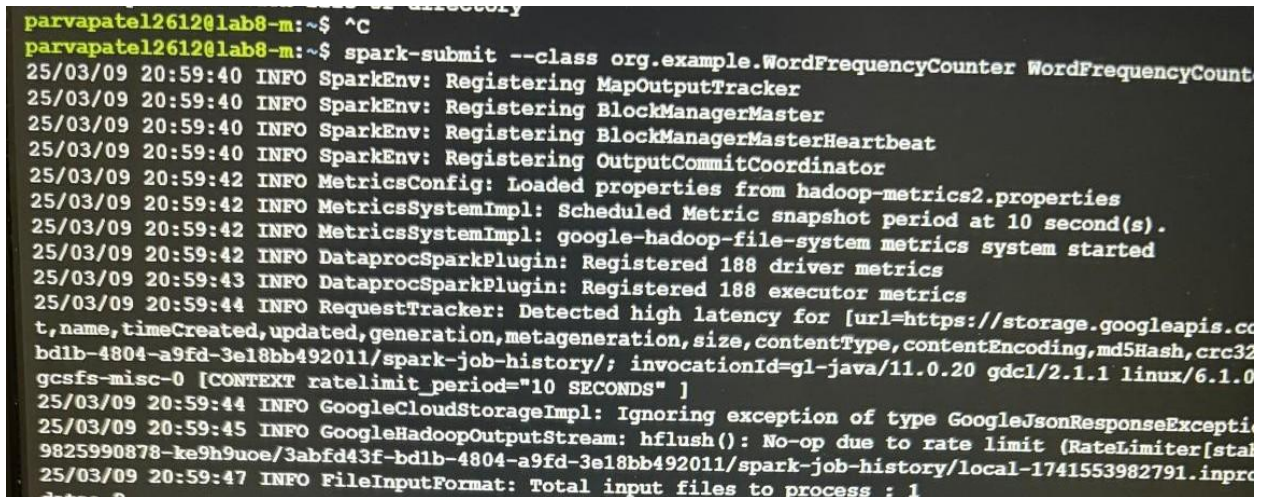


Figure 13: Sample.txt file Successfully Read

```

Last login: Sun Mar  9 19:36:37 2025 from 35.235.244.33
parvapatel2612@lab8-m:~$ cat /home/parvapatel2612/output.txt
cat: /home/parvapatel2612/output.txt: Is a directory
parvapatel2612@lab8-m:~$ nano /home/parvapatel2612/output.txt
parvapatel2612@lab8-m:~$ cat /home/parvapatel2612/output.txt/part-00000
data: 2
spark: 2
and: 2
parvapatel2612@lab8-m:~$ cat /home/parvapatel2612/output.txt/part-00001
scala: 1
learning: 1
provides: 1
is: 1
executing: 1
python: 1
simple: 1
singlenode: 1
apache: 1
science: 1
machines: 1
java: 1
engine: 1
r: 1
a: 1
on: 1
or: 1
apis: 1
in: 1
multilanguage: 1
engineering: 1
clusters: 1
machine: 1
parvapatel2612@lab8-m:~$ █

```

Figure 14: Word Count and sorted from highest to lowest

Challenges Faced while executing the JAR file on the Apache Spark cluster

A challenge I faced while executing the JAR file on an Apache Spark cluster is handling file paths.

5) References

1. **Google Cloud Platform (GCP) for Cluster Creation:** Google Cloud Platform, "Google Cloud Dataproc," [Online]. Available: <https://cloud.google.com/dataproc>. [Accessed: March 09, 2025].
2. **IntelliJ for Java Code:** JetBrains, "IntelliJ IDEA," [Online]. Available: <https://www.jetbrains.com/idea/>. [Accessed: March 09, 2025].
3. **Apache Spark SQL and Core Dependencies:** The Apache Software Foundation, "Apache Spark," [Online]. Available: <https://spark.apache.org/>. [Accessed: March 09, 2025].
4. **JavaRDD and SparkSession (Apache Spark API):** The Apache Software Foundation, "Java API for Spark," [Online]. Available: <https://spark.apache.org/docs/latest/api/java/>. [Accessed: March 09, 2025].